

BROADCAST RECEIVER

Dr.K.Sathiyamurthy, DCSE,PTU

Broadcast Receiver Fundamentals

Building Blocks of Android

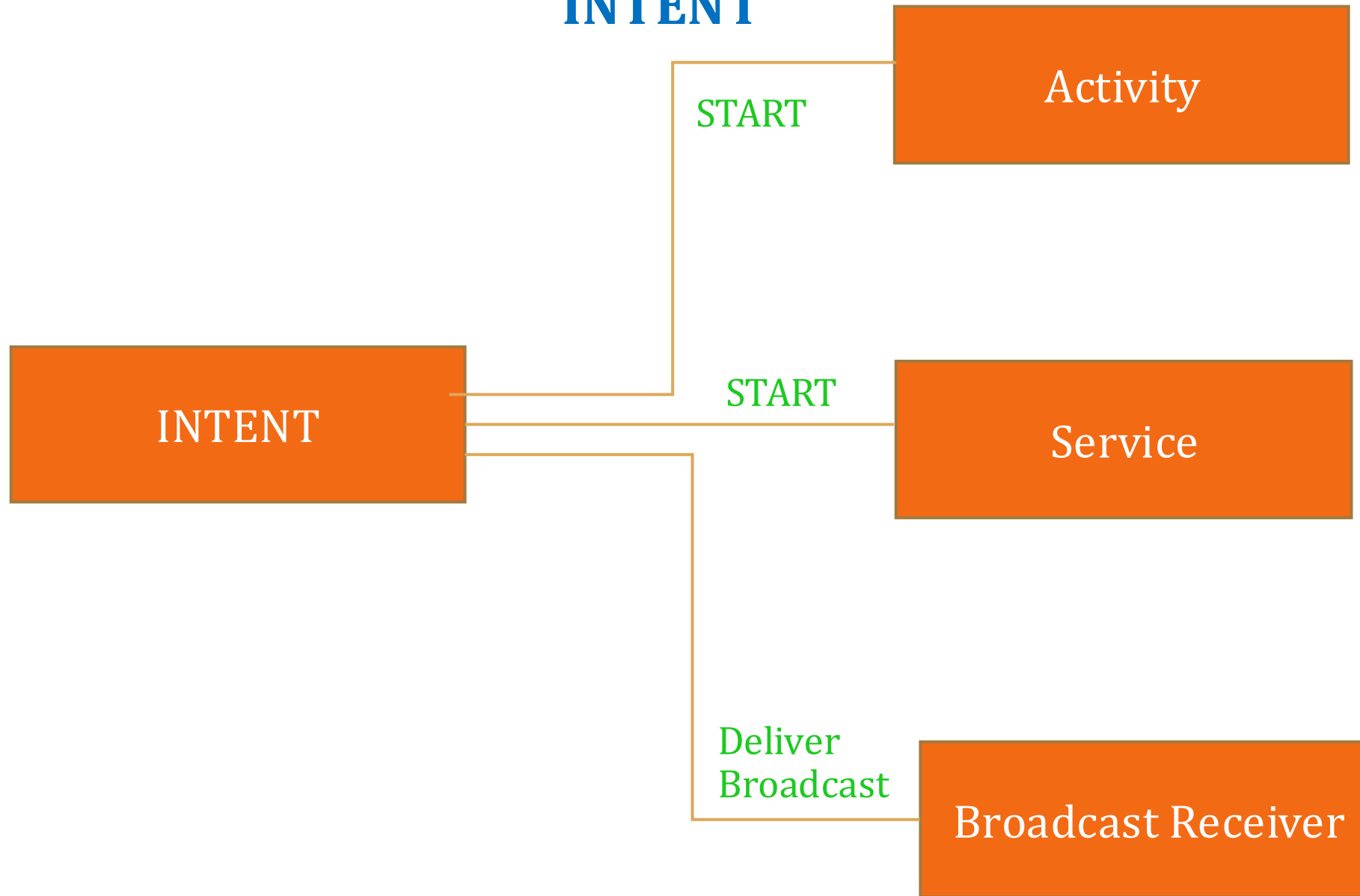
Activity

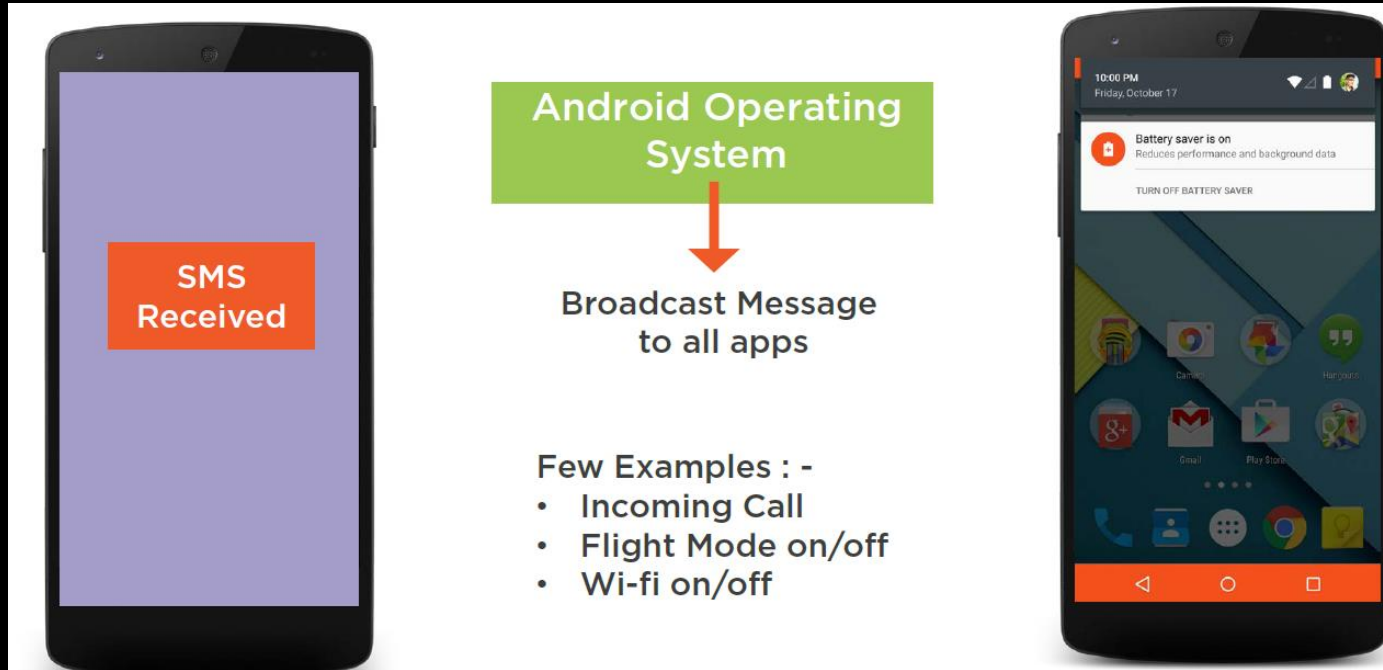
Service

BroadcastReceiver

ContentProvider

INTENT





BROADCAST RECEIVER

When a system event happens (SMS received, battery low, airplane mode, device boot), the Android OS sends a broadcast message.

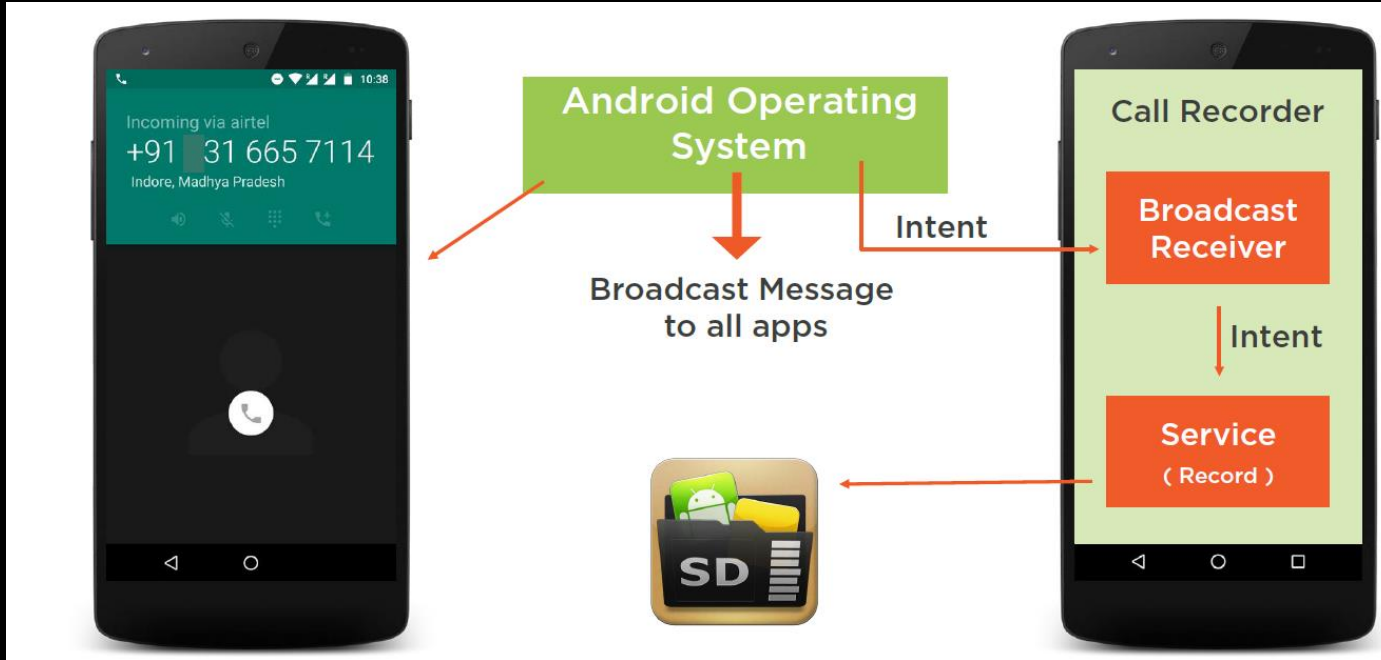
Only the apps that have registered a Broadcast Receiver for that event receive it.

Apps can also send their own (custom) broadcasts.

BROADCAST RECEIVER

Classic example: a call-recorder app registers a receiver for incoming calls; the receiver hands the recording work to a Service.

Note for today's Android: since Android 9, third-party apps cannot record call audio, and since Android 8 background service starts are restricted. Use this example only to understand the flow.



Types of Broadcast

Normal Broadcasts

`sendBroadcast()` – delivered to all receivers, in no fixed order

Ordered Broadcasts

`sendOrderedBroadcast()` – one receiver at a time, by priority (Android 16: priority only within the same app process)

Sticky Broadcasts

DEPRECATED since Android 5.0 (API 21) – do not use

Local Broadcasts

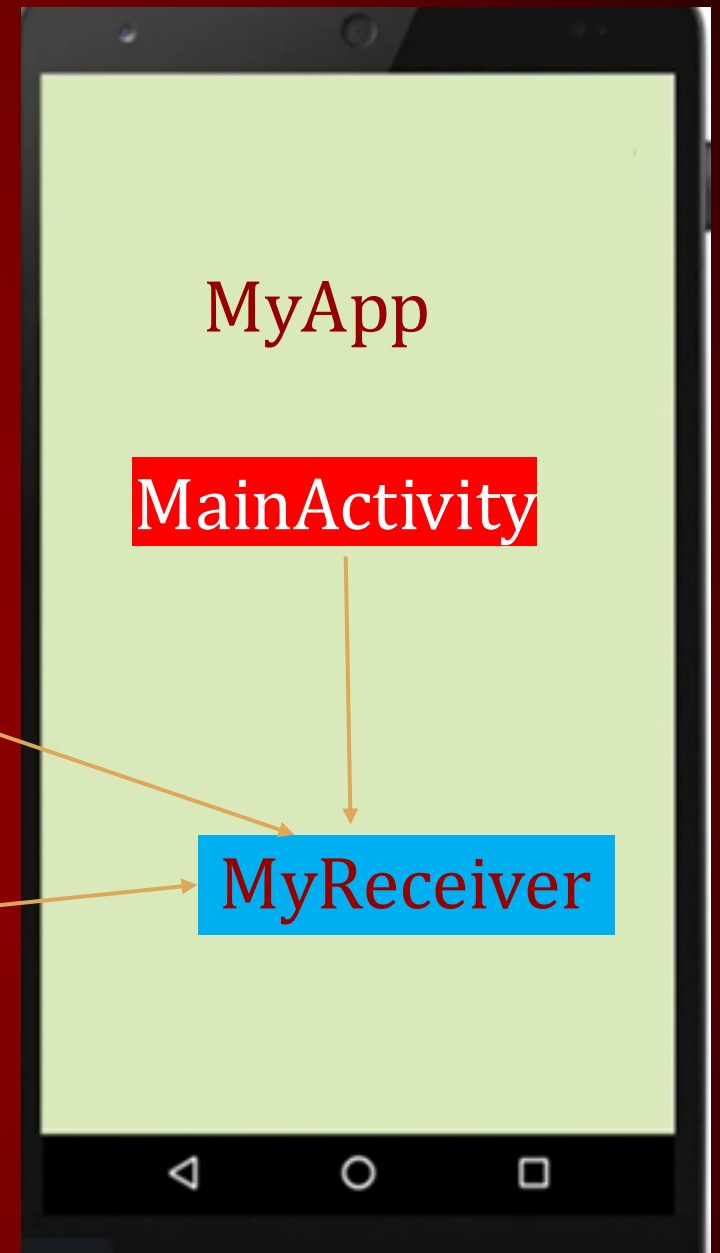
`LocalBroadcastManager` is DEPRECATED – use `LiveData` / `Kotlin Flow` / `shared ViewModel`

Also classify by sender: System broadcasts (sent by Android) and Custom broadcasts (sent by apps).

Flow of Broadcast

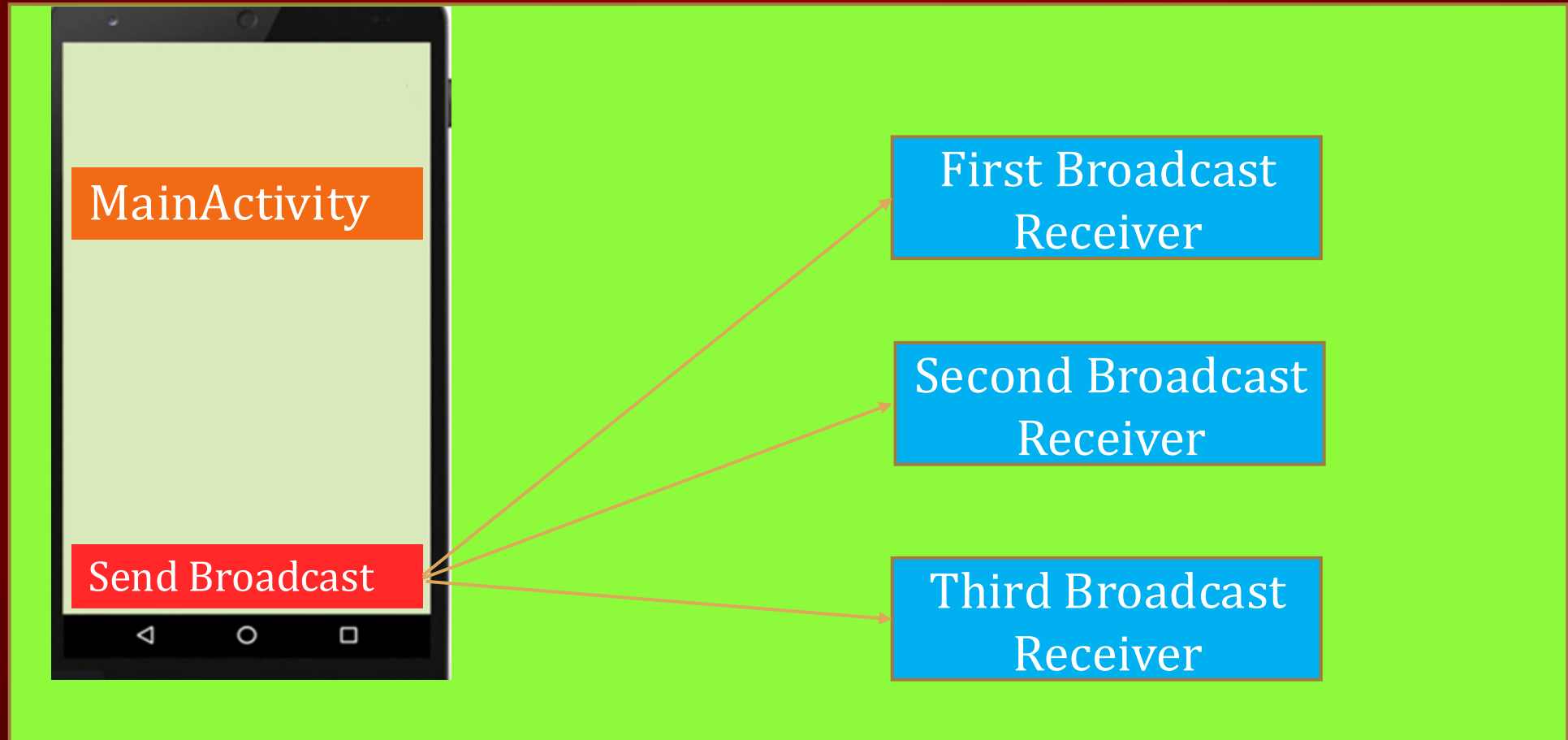
Android Operating System

1. Between OS and MyApp
Operating System -> MyReceiver
2. Between Two Apps
ABC App -> MyReceiver
3. Within the App
MainActivity -> MyReceiver



Creation of Broadcast Receiver – Statically

- ❑ Normal broadcast; all three receivers declared in AndroidManifest.xml
- ❑ Android 8+: custom broadcasts reach manifest receivers only if **explicit** – `intent.setPackage(getPackageName())`



Properties of Broadcast Receiver

- ❑ Broadcasts are delivered asynchronously – delivery time is not guaranteed
- ❑ `onReceive()` runs on the main (UI) thread
 - It must finish quickly – roughly 10 seconds
 - If it blocks too long, Android shows an ANR (“App Not Responding”) dialog for your app
 - The app’s process may be killed soon after `onReceive()` returns – don’t start threads and forget them
- ❑ **Solution (modern Android):**
 - Short async work → `goAsync()`
 - Longer / guaranteed work → `WorkManager` (plain background Services are restricted since Android 8)

Two Ways to Register a Receiver

Manifest-declared (Static)

- ❑ Declared with `<receiver>` in `AndroidManifest.xml`
- ❑ Android can start your app to deliver the broadcast, even if it is not running
- ❑ Android 8+: does NOT receive most implicit broadcasts – only exempted ones (e.g. `BOOT_COMPLETED`, `LOCALE_CHANGED`) or explicit intents
- ❑ `android:exported` must be declared (Android 12+)
- ❑ Use for: boot completed, alarms, explicit app-to-app messages

Context-registered (Dynamic)

- ❑ Registered in code with `ContextCompat.registerReceiver()`
- ❑ Works only while the Activity / app context is alive
- ❑ Receives implicit broadcasts (e.g. `AIRPLANE_MODE_CHANGED`, `BATTERY_CHANGED`)
- ❑ Must pass `RECEIVER_EXPORTED` or `RECEIVER_NOT_EXPORTED` (required when targeting Android 14+)
- ❑ Always call `unregisterReceiver()` – otherwise memory leak

Code: Manifest-declared Receiver (Boot Completed)

BootReceiver.java

```
public class BootReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        if (Intent.ACTION_BOOT_COMPLETED.equals(intent.getAction())) {
            // Keep onReceive() short - hand longer work to WorkManager
            WorkManager.getInstance(context)
                .enqueue(OneTimeWorkRequest.from(SyncWorker.class));
        }
    }
}
```

AndroidManifest.xml

```
<uses-permission
    android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>

<receiver android:name=".BootReceiver"
    android:exported="false">
    <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED"/>
    </intent-filter>
</receiver>
```

Why this still works today

- ❑ BOOT_COMPLETED is on the exemption list, so manifest receivers still get it
- ❑ android:exported is mandatory (Android 12+)
- ❑ Needs the RECEIVE_BOOT_COMPLETED permission

Code: Context-registered Receiver (Airplane Mode)

MainActivity.java

```
public class MainActivity extends AppCompatActivity {

    private final BroadcastReceiver airplaneReceiver =
        new BroadcastReceiver() {
            @Override
            public void onReceive(Context c, Intent intent) {
                boolean on = intent.getBooleanExtra("state", false);
                Toast.makeText(c, "Airplane mode: " + on,
                    Toast.LENGTH_SHORT).show();
            }
        };

    @Override protected void onStart() {
        super.onStart();
        IntentFilter f = new IntentFilter(
            Intent.ACTION_AIRPLANE_MODE_CHANGED);
        ContextCompat.registerReceiver(this, airplaneReceiver,
            f, ContextCompat.RECEIVER_EXPORTED);
    }

    @Override protected void onStop() {
        super.onStop();
        unregisterReceiver(airplaneReceiver);
    }
}
```

Key points

- ❑ Register in `onStart()`, unregister in `onStop()` – smallest scope, no leaks
- ❑ `ContextCompat.registerReceiver()` works on all Android versions
- ❑ **Export flag (Android 14+):**
 - `RECEIVER_EXPORTED` – system / other apps may send
 - `RECEIVER_NOT_EXPORTED` – only your own app (safer)
- ❑ Airplane-mode change is an implicit broadcast, so it must be registered in code, not in the manifest

Sending a Custom Broadcast – the Safe Way

Sender (e.g. MainActivity.java)

```
Intent intent = new Intent("cse.pec.demo.ACTION_DOWNLOAD_DONE");
intent.setPackage(getPackageName()); // explicit → delivered on Android 8+
intent.putExtra("file", "report.pdf");

sendBroadcast(intent); // normal broadcast
sendBroadcast(intent, "cse.pec.demo.permission.NOTIFY"); // only receivers holding this permission
sendOrderedBroadcast(intent, null); // ordered broadcast
```

Good practice

- ☐ Use `setPackage()` / `setComponent()` to target a specific app
- ☐ Protect with a permission if other apps must not read it
- ☐ Never put passwords or personal data in implicit broadcasts

Within the same app?

- ☐ Don't broadcast to yourself – it is slow inter-process messaging
- ☐ Use LiveData, Kotlin Flow or a shared ViewModel instead

What Changed in Recent Android Versions

5.0 · API 21	Sticky broadcasts deprecated
7.0 · API 24	CONNECTIVITY_ACTION no longer delivered to manifest receivers
8.0 · API 26	Manifest receivers stop getting most implicit broadcasts; background service limits
12 · API 31	android:exported must be declared on receivers with intent filters
13 · API 33	RECEIVER_EXPORTED / RECEIVER_NOT_EXPORTED flags introduced
14 · API 34	Export flag required for context-registered receivers; broadcasts to cached apps may be deferred
16 · API 36	Ordered-broadcast priority respected only within the same app process
17	SMS_RECEIVED can be withheld for up to 3 hours for OTP messages meant for another app

Also: LocalBroadcastManager (AndroidX) is deprecated.

Best Practices & Modern Alternatives

Do

- ❑ Keep onReceive() short (≈10 s limit, else ANR)
- ❑ Prefer context-registered receivers; unregister them
- ❑ Always declare android:exported and the RECEIVER_* flag
- ❑ Restrict senders / receivers with permissions and setPackage()
- ❑ Never broadcast sensitive data with implicit intents
- ❑ Don't start an Activity from a receiver – show a notification

Instead of a broadcast, use...

- ❑ **Long / background work** → WorkManager
- ❑ **Events inside your app** → LiveData, Kotlin Flow, shared ViewModel
- ❑ **Network changes** → ConnectivityManager.NetworkCallback
- ❑ **Fast app-to-app calls** → Bound Service
- ❑ **Still use broadcasts for** → system events: boot, battery, airplane mode, locale, SMS